

AI-Driven Optimal Placement of Electric Vehicles Charging Stations in Kenya

Business Understanding

Business overview

Kenya is currently experiencing a significant transformation in its transportation and energy sectors. The adoption of electric vehicles (EVs) is steadily increasing, driven by several key factors. Rising fuel costs are encouraging individuals and businesses to seek alternative, more affordable transportation options, with electric vehicles offering long-term savings. According to Siemens Stiftung (2023), the Kenyan government has introduced various incentives to support the shift toward greener energy, including tax exemptions and rebates for EV purchases. This, coupled with a global push for sustainability, has positioned Kenya as a growing market for EVs, contributing to its broader environmental goals of reducing carbon emissions and improving air quality.

Despite the momentum towards EV adoption, the country's infrastructure development has not kept pace. One of the main challenges hindering the widespread use of EVs is the absence of a data-driven approach to charging station placement (Capital FM., 2024). Currently, the deployment of charging stations is largely arbitrary, with decisions often being made reactively or based on limited insights. As a result, charging infrastructure is confined to a few key areas, usually urban centers or major highways, leaving vast regions underserved.

This uneven distribution of charging stations leads to several problems that slow down EV adoption. First, the underutilization of existing charging stations occurs because they are not strategically located to meet the needs of drivers across the country. Second, many EV owners experience range anxiety the fear of running out of battery power without access to a nearby charging point because of the uncertainty around the availability of charging stations. To overcome these challenges, a more systematic, data-driven approach to planning and placing charging stations is crucial. Such an approach would ensure that the deployment of infrastructure is aligned with actual demand, optimizing the EV charging network, reducing range anxiety, and supporting the sustainable growth of the EV market in Kenya.

Problem Statement

The adoption of electric vehicles (EVs) in Kenya is gaining momentum, fueled by the rising cost of fossil fuels, government incentives, and growing environmental awareness. However, despite this progress, the lack of a well-planned and optimized EV charging infrastructure continues to be a significant barrier to achieving widespread EV adoption.

Currently, the deployment of EV charging stations in Kenya is often done without the use of data-driven insights or strategic planning. As a result, the existing charging stations are placed in locations that may not align with the actual needs of EV owners, leading to several challenges. Many charging stations are located in areas with limited traffic or low demand, resulting in underutilization. This not only wastes valuable resources but also leads to inefficient infrastructure investment.

Project Objectives

1. Primary Objective:

To develop an AI-powered platform that uses machine learning, geospatial data, and predictive analytics to identify optimal locations for electric vehicle (EV) charging stations.

2. Secondary Objectives:

1. To integrate machine learning algorithms for analyzing EV usage patterns and infrastructure needs.
2. To incorporate geospatial data for determining the most efficient and accessible sites for charging stations.
3. To ensure the platform is scalable and adaptable to various regions or cities.
4. To improve the overall efficiency of EV charging infrastructure deployment and usage.

Approach Methodology

The approach follows a structured process that includes data preprocessing, feature extraction, model selection, and evaluation. Through an iterative process of testing and refining various techniques, the aim is to accurately identify geographic clusters of charging stations by modeling data from a U.S. city with the highest concentration of charging stations. This methodology will generate valuable insights that can be applied to map optimal charging station locations in Nairobi, ensuring a more strategic deployment of infrastructure in the city.

To address the project objectives, the methodology follows a structured process:

1. **Data Cleaning:** Start by handling missing values, removing duplicates, standardizing data types, and managing outliers. This ensures that the dataset is accurate and ready for analysis.
2. **Feature Engineering:** Create new features that can improve the performance of clustering models. This step is important for enhancing the model's ability to identify meaningful patterns within the data.
3. **Data Scaling and Normalization:** Since clustering models like K-means are sensitive to the scale of the data, it is essential to standardize the features. Techniques like Min-Max scaling ensure that all features contribute equally to the distance calculations. Also check for correlations between features to determine if dimensionality reduction is necessary.
4. **Choosing the Number of Clusters:** The elbow method is used to determine the optimal number of clusters (K). This method plots the within-cluster sum of squares (WCCS) against the number of clusters and identifies the "elbow" where the rate of improvement slows down. The silhouette score is then calculated to validate the chosen K.
5. **Modeling:** Various clustering algorithms, including K-means, DBSCAN, and Gaussian Mixture Models (GMM), are applied. Each model is initialized with the optimal K value from the Elbow plot and then fitted to the data.
6. **Model Evaluation and Results Interpretation:** After fitting the model, evaluate the cluster centers, assign labels to the data points, and visualize the clusters. If necessary, refine the model by adjusting the number of clusters (K) and reassessing the data point assignments.

7. Application to Nairobi: Once the model is trained and refined, it is applied to a new dataset from Nairobi. The trained clustering model, which has been learned from another city, is used to map potential EV charging station locations in Nairobi, allowing insights from one area to inform decisions in another.

Metrics of Success.

To assess the effectiveness of the cluster models, the following evaluation metrics will be used:

Prediction Accuracy for High-Demand Locations:

The model should correctly predict at least 90% of high-demand locations, minimizing false positives (incorrectly predicting high demand) and false negatives (failing to predict high demand) when identifying optimal sites for charging stations.

R² Score:

The model should achieve an R² score of at least 0.85, indicating a strong correlation between the predicted charging demand and observed actual demand. This ensures the model's reliability in estimating demand.

Proximity to Power Grid:

At least 80% of the suggested charging station locations should be within 500 meters of a power grid connection, ensuring that the proposed sites are practical for deployment and accessible to the existing energy infrastructure.

Adaptability to New Urban Areas:

The model should maintain an accuracy above 85% when tested on new urban areas, demonstrating its ability to adapt as Nairobi's EV market expands and ensuring that it is not over-fitted to a specific dataset but can generalize well to new locations.

Data Understanding

Obtaining the data

The data for this project was sourced from the U.S. Department of Transportation, Bureau of Transportation Statistics, specifically focusing on Electric Vehicle Public Charging Stations in the United States as of January 2020. The dataset provides comprehensive information about the locations and characteristics of EV charging stations across the U.S., serving as the basis for clustering and predictive modeling.

You can access the dataset via the following link:

[Electric Vehicle Public Charging Stations Data](#)

We read the dataset in pandas as shown below:

```
#Alternative_Fueling_Stations.csv

# Load the first sheet into a DataFrame
df = pd.read_csv('Alternative_Fueling_Stations.csv')

# Show the shape of the data
print(df.shape)
# Display the first few rows to understand the structure
df.head()
```

```
(97882, 80)
```

Fig.1: Loading the data.

This dataset will identify patterns and apply clustering models to predict optimal charging station locations, which can later be adapted for use in Nairobi or other cities with similar infrastructure needs.

Data Preparation

1. Filtering the dataset tailored for our analysis
2. Data cleaning
3. Data preprocessing: feature engineering transformation and preprocessing.

The dataset provided information about alternative fueling stations, including Electric Vehicle (EV) charging stations. It consisted of 97,882 rows and 80 columns. Key columns relevant to EV charging stations include:

- Access code: Indicates whether the station is public or private.
- Access_days_time: Specifies availability, such as 24-hour access.
- Fuel_type_code: Identifies the fuel type (e.g., "EV" for Electric Vehicles).
- Groups_with_access_code: Details the access permissions.
- Ev_pricing_fr: Describes the pricing model for EV charging.
- Ev_network_ids_station: Identifies the EV charging network.
- X, Y: Longitude and latitude coordinates for mapping the station location.
- Federal_agency_name: Indicates whether the station is government-owned or privately owned.

We began by filtering the dataset to include only rows with the 'ELEC' fuel_type_code, focusing on Electric Vehicle (EV) charging stations.

```
df_elec = df[(df['fuel_type_code'] == 'ELEC')]  
df_elec.shape
```

```
df_elec['state'].value_counts().head(10)
```

```
df_elec['city'].value_counts().head(10)
```

Fig.2: Filtering the Data frame.

Next, we identified the features most relevant to our analysis and filtered the data frame to retain only those columns. To ensure the precision of our analysis, we further filtered the dataset to include only the 'ELEC' fuel type and then narrowed it down to the state with the highest count of EV stations in the filtered data. California emerged as the state with the highest number of EV charging stations, and Los Angeles had the highest concentration of EV stations within California. Therefore, we filtered the data to focus on Los Angeles, ensuring we examined the most pertinent subset of the dataset for our analysis

```
df_elec = Ev_df_noduplicate.copy()  
df_elec = df_elec[(df_elec['state'] == 'CA') &  
                  (df_elec['city'] == 'Los Angeles')]  
df_elec.shape
```

Fig.3: Filtering the Data frame.

The filtered dataset now contained only Electric Vehicle (EV) charging stations located in California, specifically in Los Angeles. While the dataset was primarily focused on EV stations, it still included columns related to other fuel types such as LPG and ethanol. To refine the dataset further, we removed these irrelevant columns, retaining only those that pertain to EV charging stations. To ensure we had the correct and relevant columns, we created a list of these columns and used the check info function from the my functions directory to examine the dataset's structure and confirm the data's accuracy and relevance. This allowed us to verify that only the necessary columns were retained for further analysis.

```
relevant_columns = [  
    'station_name', 'latitude', 'longitude', 'city', 'state',"restricted_access",  
    "ev_renewable_source", 'street_address', 'status_code', 'access_code',"cards_accepted",  
    "ev_workplace_charging", 'ev_connector_types', 'ev_dc_fast_num', 'ev_level1_evse_num',  
    "intersection_directions", "owner_type_code", 'ev_level2_evse_num', 'ev_network',  
    'ev_pricing',"access_days_time", "maximum_vehicle_class", "facility_type"  
]  
  
Ev_df = df_elec[relevant_columns]  
Ev_df  
  
Mf.check_info(Ev_df)
```

Fig.4: Filtering the relevant columns.

The dataset now contained 1,941 records and 23 columns, providing detailed information about electric vehicle (EV) charging stations, including station names, locations (latitude, longitude, city, state), status, access details, EV-specific features (such as connector types and charging levels), and operational attributes like pricing and workplace charging options. Although the dataset focuses on EV stations, it includes missing values in several columns, particularly in areas related to renewable sources, pricing, and specific charging capabilities. The data is predominantly of object and float64 types, and there are no duplicated rows, ensuring that each record is unique.

The filtered dataset now focuses on electric vehicle (EV) charging stations in California and Los Angeles. This refinement ensures that our analysis is more targeted and relevant, eliminating any irrelevant data that could distort the results. In this section, the focus shifts to data preparation, a critical step in the analysis process. This involves cleaning and transforming the dataset to ensure it is suitable for exploratory data analysis (EDA) and modeling. Data cleaning will address missing values, duplicates, and irrelevant columns, maintaining the integrity of the analysis. Additionally, transformations will be applied to standardize data formats, handle outliers, and engineer new features, all of which are essential for improving the performance of machine learning models.

Data Cleaning

1. Deal with the missing values and filtering for important columns.
2. Deal with Duplicates
3. Further cleaning and transformation; creating new columns, encoding features etc.

Dealing with Missing Values

We identified missing entries across the dataset, and we will use the `.isna()` method. This method calculates the percentage of missing values for each column. The results will then be sorted in descending order to highlight the columns with the most missing data. The final output will be displayed as a data frame, showing each column and the corresponding percentage of missing entries.

```
# Dropping columns with above 10% missing values
def drop_high_null_columns(df, threshold=20):
    return df.loc[:, (df.isna().sum() / len(df) * 100) <= threshold]

df_20_percent=drop_high_null_columns(Ev_df, threshold=20).reset_index()
df_20_percent.head(3)
```

```
Ev_df1 = df_20_percent.copy()
```

Fig.5: Dropping columns with more than 10% missing.

The filtered EV Charging Station Data set now included several key categories of information.

station_name: The name of the EV charging station typically reflects the location or brand.

Latitude: The north-south geographic coordinate of the station (in degrees).

Longitude: The east-west geographic coordinate of the station (in degrees).

City: The city where the EV charging station is located.

State: The state or region where the station is located (if applicable).

Country: The country where the EV charging station is located.

street_address: The full street address, including building number and street name.

status_code: Indicates the station's current status (e.g., "Available," "Out of Service").

access_code: A code or identifier required to access the charging station (e.g., PIN, card number).

ev_connector_types: The types of connectors available (e.g., Type 1, Type 2, CCS, CHAdeMO).

ev_dc_fast_num: The number of DC fast chargers that provide rapid charging.

ev_level1_evse_num: The number of Level 1 EVSE units offering slow charging (typically residential).

ev_level2_evse_num: The number of Level 2 EVSE units offering faster charging commonly found in public networks.

Network ev_network: The network's name operating the station (e.g., Tesla Supercharger, Charge Point).

Imputing ev_level2_evse_num with zeroes

In this step, we addressed the missing values in the ev_level2_evse_num column, which indicates the number of Level 2 electric vehicle (EV) chargers at each station. The absence of data here is not due to data collection errors but reflects that certain stations do not have Level 2 chargers. Instead of imputing these missing values with a mean or median, which could artificially inflate the data and give a false impression of charger availability, we imputed these missing values with zeroes. This approach ensures that the data remains truthful and does not suggest the presence of infrastructure where none exists, maintaining the integrity of the dataset without introducing erroneous assumptions.

Dropping the access_days_time missing rows

The access_days_time column denotes the hours a charging station is operational during the day. Missing values in this column represent a lack of information about the station's operating schedule. Instead of filling in the missing entries with zeroes or a median value, implying that the station operates for a full or default period, potentially distorting the data, we dropped rows with missing values. This decision ensures the accuracy of the dataset, as it avoids making unwarranted assumptions about the operational hours of the stations. By excluding these rows, we preserve the integrity of the dataset and ensure that the analysis reflects the true, available data without introducing biases.

```
# Handling the missing values
Ev_df1['ev_level2_evse_num'] = Ev_df1['ev_level2_evse_num'].fillna(0)
# Dropping rows with missing entries in the access_day_time column
Ev_df1 = Ev_df1.dropna()
print(f"We have {Ev_df1.isna().sum().sum()} nulls in our dataframe")
```

- We now can confirm the dataframe has no null entries.

Fig.6: Imputing missing entries.

Further Data Cleaning and Feature Engineering

In this section, we split the dataset into categorical and numerical columns using the .select_dtypes() method. This allowed us to create two separate data frames for each data type. This step's main focus was cleaning the categorical columns to ensure they were ready for analysis or modeling.

Cleaning the categorical columns involved addressing several key tasks, including identifying and resolving any inconsistencies within the data, handling missing values appropriately, and

converting the variables into formats compatible with further analysis or modeling techniques. These steps are crucial because well-prepared categorical data ensures that subsequent models can effectively interpret and utilize the variables without errors or misinterpretations.

The emphasis on categorical data in this step is to ensure that these variables are properly processed and structured for modeling purposes. Properly prepared categorical data enhances the accuracy of any predictive models and ensures that any insights derived from the analysis are based on clean and reliable inputs.

```
▶ # Viewing the numerical columns
Numericals = Ev_df1.select_dtypes(include='number')
Numericals.head(3)

▶ # Viewing the Categorical Columns
categoricals = Ev_df1.select_dtypes(include='object')
categoricals.head(4)

▶ categoricals.columns
```

Fig.7: Splitting Data set into numerical and categorical.

Cleaning Categorical Columns and Preprocessing

The station_name column

To begin the cleaning process for the station_name column, we created a function named clean_and_split_station_name. This function performs a series of steps to ensure the column is properly processed for further analysis and modeling:

1. Stripping Leading and Trailing Spaces: The function removes any unnecessary spaces at the beginning or end of the station names to maintain consistency and prevent issues during analysis.
2. Splitting the String: The function splits the station_name into two parts:
 - station_region: This is the portion of the string before the hyphen (-).
 - station_subregion: This is the portion of the string after the hyphen (-). If the hyphen is missing, both new columns (station_region and station_subregion) will be filled with the same string, representing the entire station name as the region and subregion.
3. Handling Invalid Entries: If the entry is not a valid string (e.g., a numeric value or missing data), the function will return None for both the station_region and station_subregion columns to ensure that these invalid values do not interfere with further processing.
4. Frequency Encoding: To prepare these newly created columns for modeling, the function applies frequency encoding to both the station_region and station_subregion. This encoding technique assigns each category a numeric value based on how frequently that category appears in the dataset, providing a way for the model to interpret the categorical data numerically.

The function was then called from the Python file containing custom functions in the project directory to apply these transformations to the station_name column. This preprocessing ensures the data is in a suitable format for further modeling and analysis.

```
# Cleaning and encoding the station_name columns
def clean_and_split_station_name(station_name):
    """
    Cleans the station name by stripping extra spaces and splitting it into two columns:
    - 'station_region': The part before the hyphen
    - 'station_subregion': The part after the hyphen

    Args:
        station_name (str): The station name.

    Returns:
        tuple: (station_region, station_subregion)
    """
    if isinstance(station_name, str):
        parts = station_name.strip().split(' - ', 1)
        return parts if len(parts) == 2 else (station_name, station_name)
    return None, None # Handle NaN or non-string values

# Apply the function to split the column efficiently
Ev_df2[['station_region', 'station_subregion']] = (
    Ev_df2['station_name']
    .astype(str) # to convert everything to string to avoid errors
    .str.strip()
    .str.split(' - ', n=1, expand=True)
    .fillna(method='ffill', axis=1) # Fill second column with first if there's no hyphen
)
Ev_df2[['station_region', 'station_subregion']]
Ev_df2['station_region_encoded'] = Ev_df2['station_region'].factorize()[0]
Ev_df2['station_subregion_encoded'] = Ev_df2['station_subregion'].factorize()[0]
Ev_df2.head(2)
```

Fig.8: Function to clean the station_name column.

The access_days_time Column

To clean and process the access_days_time column, we developed a function called calculate_hours. This function calculated the number of operational hours for each entry based on the start and end times extracted from the column. The function ensured that the end time was always greater than the start time, addressing any inconsistencies in the data.

The calculate_hours function included an internal function, extract_time_and_hours, which is responsible for extracting time details from the parking lot access descriptions. This internal function handled various time formats, including:

1. "24 hours" Descriptions: For descriptions that indicated "24 hours," the function automatically assigned a value of 24 hours, representing continuous access throughout the day.
2. Day-Specific Time Ranges: The function processed entries with specific days and time ranges (e.g., "Mon 9:00 am - 5:00 pm"). It extracted the start and end times, converted them into a 24-hour format, and calculated the total number of hours between the two times.

The function also ensured that if the end time was earlier than the start time, indicating a potential error in the data, it handled the situation by adjusting the times or flagging the entry for review.

After processing the time descriptions, we used the process_dataframe function to extract and calculate the time and hours from the access_days_time column. This resulted in adding a new column, access_hours, to the dataset, which stored the calculated number of hours each parking lot was accessible.

This process standardized the access_days_time column, ensuring the data was consistent and reliable for analysis. It allowed us to calculate the total access hours for each parking lot, which simplified further analysis and modeling efforts.

```

# Cleaning the access daytime column
def calculate_hours(start_time: str, end_time: str) -> float:
    """
    Calculate the number of hours between start and end time.
    """
    start = datetime.strptime(start_time, "%H:%M")
    end = datetime.strptime(end_time, "%H:%M")

    if end < start:
        end = end + timedelta(days=1)

    diff = end - start
    hours = diff.total_seconds() / 3600

    return round(hours, 2)

def extract_time_and_hours(text: str) -> dict:
    """
    Extract time information and calculate hours from parking lot access descriptions.
    """
    # Handle non-string input

```

Fig.8: Function to access_day_time column.

The ev_connector_types Column

To clean and preprocess the ev_connector_types column, we first addressed the entries consisting of lists of charger types. The cleaning process involved several key steps:

1. Removing Unwanted Quotes: We replaced any double quotes (") in the entries with single quotes ('), ensuring uniformity in the data.
2. Stripping Leading and Trailing Whitespaces: We removed any unnecessary leading or trailing whitespace from the column values to ensure formatting consistency.

Next, we created a function that converted any string representations of lists into actual Python lists using the ast.literal_eval function. This function was applied only to string values, as other data types (e.g., already formatted lists) did not require conversion.

After applying this function to the Ev_df2 DataFrame, the ev_connector_types column was properly formatted as Python lists.

We then performed one-hot encoding for the cleaned column to prepare the data for modeling.

We created a function named encode_connector_types that carried out the following steps:

1. Checking for Lists: The function first checked that each entry in the ev_connector_types column was a list.
2. Handling Missing or Non-List Entries: It accounted for missing values or entries that were not lists, ensuring that such cases were appropriately managed and did not cause errors during encoding.
3. Encoding with MultiLabelBinarizer: The function used the MultiLabelBinarizer from the sklearn.preprocessing module to perform the one-hot encoding. This transformed the lists of charger types into multiple binary columns, each representing a specific EV connector type.
4. Concatenating the Encoded Columns: The newly created binary columns were then concatenated with the original data Frame.
5. Dropping the Original Column: After encoding, the original ev_connector_types column was dropped from the data Frame, as it was no longer necessary.

Finally, we applied the encode_connector_types function to the Ev_df2 data Frame, and the function returned the resulting data frame with the newly encoded columns.

This process ensured that the ev_connector_types column was properly cleaned, encoded, and integrated into the data frame for further analysis and modeling.

```

# Cleaning the ev_connector_types column
# Function to clean and parse the ev_connector_types column
def clean_ev_connector_types(df, column_name="ev_connector_types"):
    # Strip spaces and remove unwanted quotes
    df[column_name] = df[column_name].str.replace(r'""', '', regex=True).str.strip()
    # Convert string lists to actual Python lists
    df[column_name] = df[column_name].apply(lambda x: ast.literal_eval(x) if isinstance(x, str) else x)
    return df
# Apply function
Ev_df2 = clean_ev_connector_types(Ev_df2)
Ev_df2.tail(3)

def encode_connector_types(df, column_name='ev_connector_types'):
    """
    One-hot encode a column containing lists of EV connector types.

    Parameters:
    df (pandas.DataFrame): DataFrame containing the connector types column
    column_name (str): Name of the column containing lists of connector types

    Returns:
    pandas.DataFrame: DataFrame with encoded connector types
    """
    from sklearn.preprocessing import MultiLabelBinarizer
    import pandas as pd

    # Create a copy to avoid modifying the original DataFrame
    df_encoded = df.copy()

    # Ensuring the column contains lists (handle missing values and non-list entries)
    df_encoded[column_name] = df_encoded[column_name].apply(
        lambda x: x if isinstance(x, list) else [] if pd.isna(x) else [x]
    )

    # Instantiating MultiLabelBinarizer
    mlb = MultiLabelBinarizer()

```

Fig.9: Function to clean the ev_connector_types column.

The status_code Column

The status_code column contains three entries: E, T, and P. One-hot encoding was the most appropriate method for handling this column. This technique converts the unique values in the status_code column into separate binary columns, making it suitable for modeling.

We used the PD.get_dummies function from the pandas library to perform this encoding. This function automatically transforms the unique values in the status_code column into individual binary columns. Each new column represents one of the unique status codes, where:

- A 1 indicates the presence of that specific status code for the respective row.
- A 0 indicates its absence.

As a result, the original status_code column was replaced with multiple binary columns, each corresponding to a different status code (E, T, and P). This transformation ensured that the status_code column was properly formatted for machine learning models. Each status code is now represented as a distinct feature with binary values.

```

# Check the value counts of column status code
Ev_df2['status_code'].value_counts()

# Apply one-hot encoding
Ev_df2 = pd.get_dummies(Ev_df2, columns=['status_code'])
# Display the dataframe
Ev_df2.head(3)

```

Fig.10: Encoding the status_code.

The access_code Column

To clean the access_code column, which contains binary entries ("private" and "public"), we followed these steps:

1. Convert to Lowercase: We first converted all the values in the access_code column to lowercase to ensure consistency and handle any discrepancies in capitalization.
2. Remove Leading and Trailing Whitespace: Any unwanted leading or trailing whitespace was removed from the values to ensure that the data did not suffer from unnecessary spaces.
3. Map to Binary Numeric Values: The cleaned values were then mapped to binary numeric values:
 - "private" was mapped to 0
 - "public" was mapped to 1
4. Store in New Column: The cleaned, mapped values were stored in a new column called access_code_cleaned.

This process ensured that the access_code column was properly cleaned and converted into a numerical format more suitable for analysis and modeling.

```
pd.set_option('display.max_columns', None)
# Cleaning the Access code column
Ev_df2["access_code_cleaned"] = Ev_df2["access_code"].astype(str).str.lower().str.strip().map({"private": 0, "public": 1})
Ev_df2.tail(3)
```

Fig.11: Encoding the access_code.

The ev_network Column

To clean and process the ev_network column, which represents the type of chargers or the manufacturer, we followed these steps:

1. Stripping Leading and Trailing Spaces: We removed any leading or trailing spaces from the entries in the ev_network column to ensure that no unnecessary whitespace would interfere with further analysis.
2. Applying Frequency Encoding: We then applied frequency encoding to the column. In this step, each unique value in the ev_network column was mapped to its corresponding frequency, representing the number of times that value appeared in the column.
3. Storing in a New Column: The calculated frequencies were stored in a new column called ev_network_encoded.

This approach ensured that the ev_network column was properly cleaned and encoded, with each unique charger type or manufacturer now represented by its occurrence frequency. The new column, ev_network_encoded, was now suitable for analysis or modeling.

```
# Clean the ev_network column (remove extra spaces, unify "Non-Network" with "Non-Worked")
Ev_df2["ev_network"] = Ev_df2["ev_network"].str.strip()

# Perform frequency encoding
freq_encoding = Ev_df2["ev_network"].value_counts().to_dict()
Ev_df2["ev_network_encoded"] = Ev_df2["ev_network"].map(freq_encoding)

pd.set_option('display.max_columns', None)
Ev_df2.head(2)
```

Fig.12: Frequency encoding the ev_network.

Exploratory Data Analysis (EDA)

Univariate Analysis

In this section, we conducted a univariate analysis to examine each variable (or feature) individually, aiming to understand its distribution, spread, and central tendency. The goal was to analyze one column at a time, focusing on key statistical and visual measures to assess the nature of the data.

For Numerical Columns:

We started by evaluating the descriptive statistics for each numerical column, which included:

- Mean: The average value of the data, providing a central tendency measure.
- Median: The middle value is less sensitive to outliers than the mean.
- Mode: The most frequently occurring value, highlighting any repeated data points.
- Standard Deviation: A measure of the spread or dispersion of the data, indicating how much the values deviate from the mean.
- Percentiles: Specific values representing the distribution of the data, such as the 25th, 50th, and 75th percentiles, provide insight into the spread and distribution.

Visualizations for Numerical Data:

To better understand the numerical columns visually, we used the following plots:

- Histograms: These were used to assess the distribution of the numerical data, showing how the data points are spread across different ranges.
- Box Plots helped us detect outliers. They showed the distribution of data based on quartiles and highlighted any extreme values that might require further examination.
- Density Plots: These visualized the probability distribution of the numerical data, providing a smooth representation of how data points are likely to be distributed.

For Categorical Columns:

For categorical variables, the analysis focused on summarizing the frequency of different categories within each column:

- Frequency Counts show how often each category appears in the dataset, giving an idea of the distribution of categories. Bar Charts: We used bar charts to represent the frequency distributions of categorical data, visually displaying the count of each category and helping to highlight any imbalances or trends in the data.

This univariate analysis helped us better understand the individual features, their distributions, and potential issues such as outliers, skewed distributions, or imbalances in categorical data. This information informed further steps in our data processing and modeling.

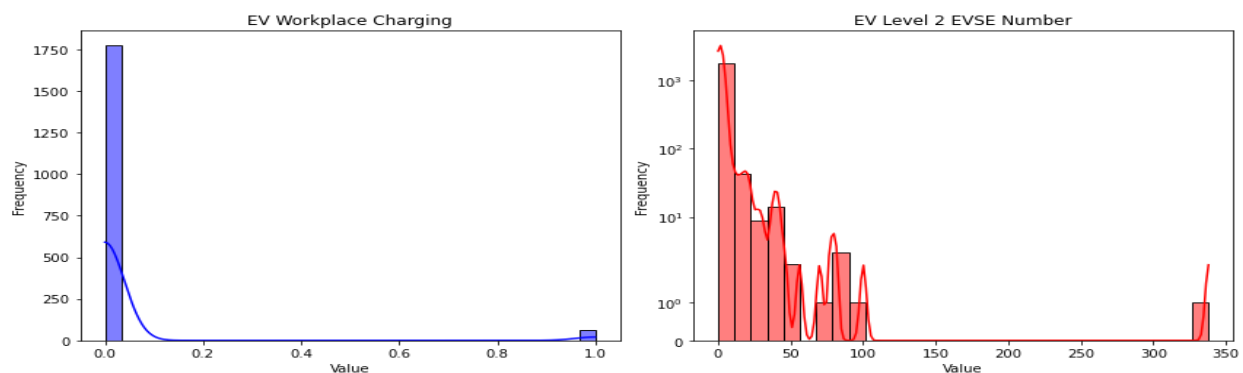


Fig.12: Univariate Analysis for Numerical Features.

Both `ev_level2_evse_num` and `ev_workplace_charging` show positive skewness, with a longer right tail in their distributions.

1. `ev_level2_evse_num` (Level 2 Chargers per Station): Most stations have only a few chargers, but a few stations have many, creating the right tail. This indicates that most stations are less equipped, with outliers representing larger or more advanced stations.
2. `ev_workplace_charging` (Workplace Charging Availability): The positive skew suggests that most workplaces do not offer charging facilities, with only a small number providing them.

Both columns indicate a concentration of lower values, with a few higher values stretching the distribution, highlighting infrastructure imbalances.

Univariate Analysis for Categorical Features.

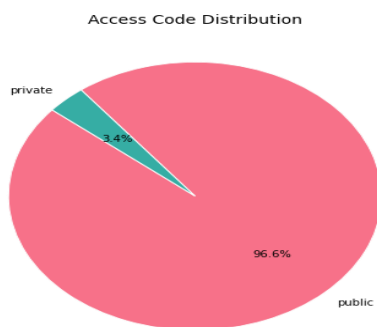


Fig.13: Distribution of the access code.

The distribution of the access code column revealed an uneven class distribution, with stations classified as private accounting for only 3.4% of the data, while public stations made up 96.6%. This imbalance posed a challenge for predictive modeling, as most stations were classified as public. This imbalance could lead to biased results in the context of clustering, particularly with the K-means algorithm. Since K-means is sensitive to the data distribution and tends to favor the majority class, the model might predominantly classify stations as public, potentially overlooking important patterns related to private stations. Addressing this imbalance was crucial to ensure the clustering model produced meaningful and representative clusters.

To tackle this issue, a class weighting strategy was employed. Applying class weighting ensured that the algorithm gave more attention to the minority class, thereby mitigating the bias towards the majority class. This approach improved the clustering process by encouraging the model to treat both classes more equally. Using sample weights proved particularly beneficial for several reasons: it prevented the minority class from being overshadowed by the majority class, maintained the integrity of the original data without altering its structure, and often led to more balanced and insightful clusters, thus improving the model's effectiveness.

Additionally, the `ev_workplace_charging` column from the numerical section displayed an imbalance between EV stations classified as 1 and 0. To address this, the same function was applied to eliminate the class imbalance, further refining the model's performance and ensuring a better representation of both classes.

The ev network column

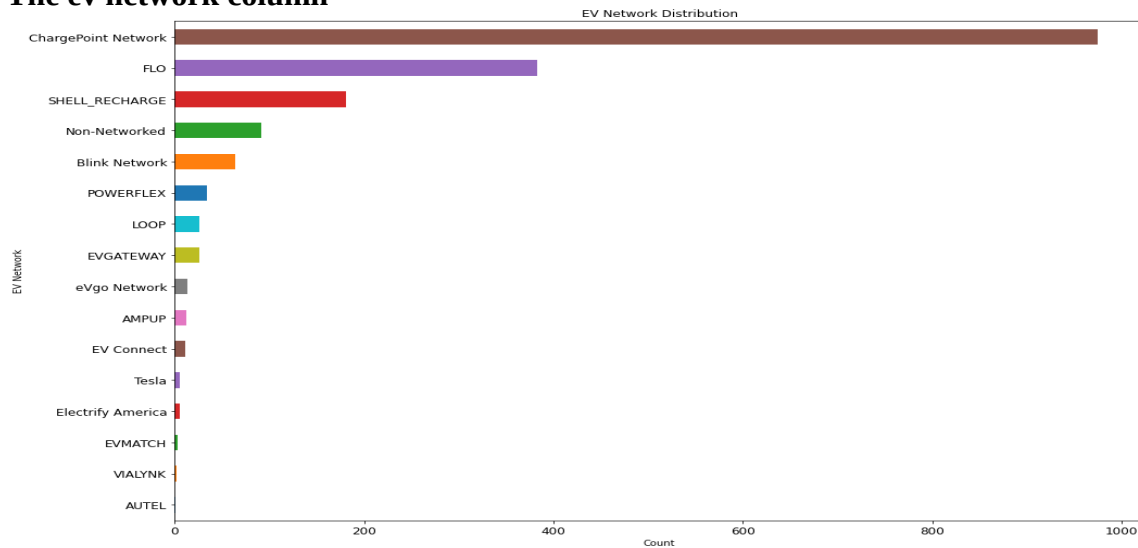


Fig.14: Distribution ev network.

The Charge Point network leads with almost 1,000 EV charging stations, followed by FLO stations, Shell Recharge, Blink Network, and Non-Networked stations. Together, these providers dominate the public charging landscape, significantly influencing the accessibility of EV infrastructure. On the other end of the spectrum, Tesla, EV Connect, AMPUP, Electrify America, EVMATCH, VIALYNK, and AUTEL have the fewest stations, with some networks operating as few as a single location.

Bivariate Analysis

In this section, a bivariate analysis examines the relationship between two variables simultaneously. The goal is to understand how these variables interact and identify important correlations or patterns. The focus is on detecting potential dependencies or associations between pairs of numerical and categorical features.

For numerical variables, the analysis includes:

- Correlation and heat-maps to quantify the relationship between variables.
- Scatter plots to visualize the strength and direction of the relationship.
- Box plots to explore multiple variable relationships in a single view.

For categorical variables, the analysis involves:

- Contingency tables to examine the frequency distribution between categories.
- Stacked bar charts to visualize how categories in one variable are distributed across another.

The ev_workplace_charging' by 'ev_level2_evse_num

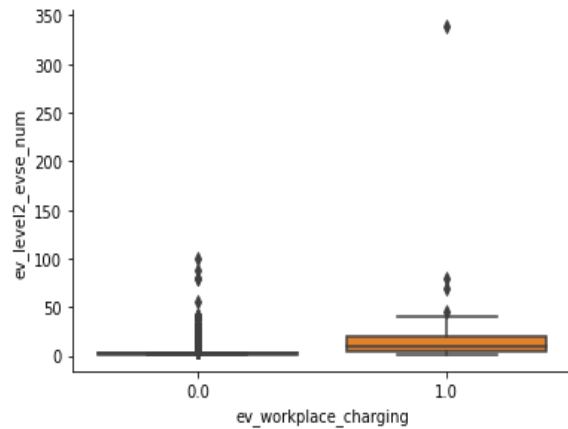


Fig.14: Box plot: ev workplace charging by ev level evse num.

The box plot visualization for the `ev_workplace_charging` column reveals the presence of an outlier. This suggests an EV station with a significantly higher number of `ev_level2_evse_num` chargers than the rest, indicating an anomaly or an exceptional case within the dataset. This special case could alter the clustering model's performance. It is recommended that the outlier be removed to improve the model's accuracy and prevent skewed results.

Charging ports by Number of Stations

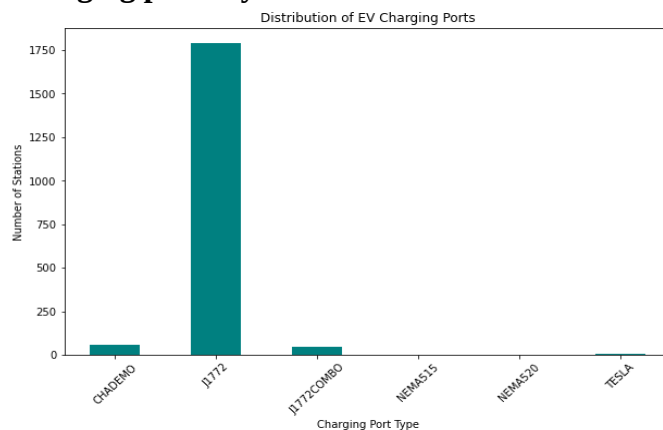


Fig.16: Bar plot: Charging ports by number of stations.

The J1772 connector is the most widely used for Level 2 charging, available at over 1,750 stations. Following this, the J1772 Combo and CHAdEMO connectors are found at fewer than 50 stations, primarily supporting fast charging. In contrast, NEMA connectors (520 and 515) and Tesla chargers are less common in public charging networks. As electric vehicle adoption grows, the J1772 connector remains the standard for Level 2 charging. At the same time, NEMA and Tesla's outlets will likely serve as secondary options rather than mainstream solutions in the EV infrastructure landscape.

Number of Stations by Access hours

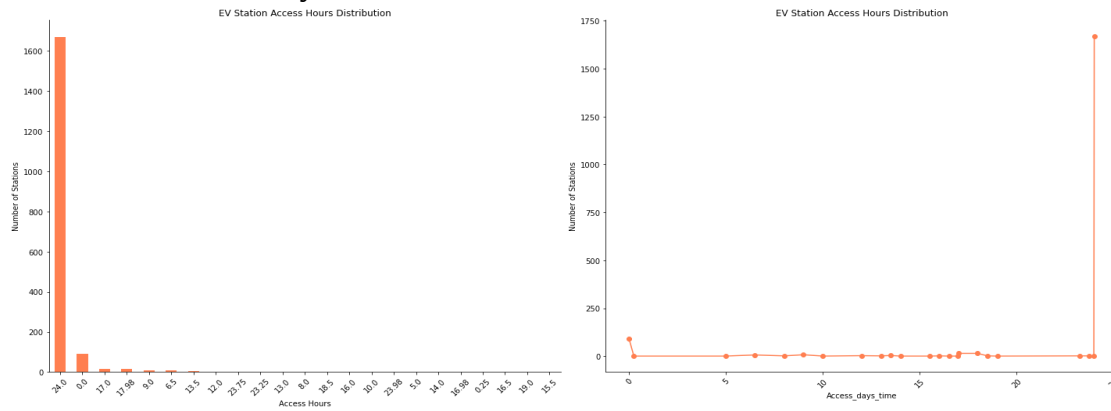


Fig.17: Bar plot and line graph: Access hours by number of stations.

In Los Angeles, more than 1,600 EV charging stations are accessible 24 hours daily. However, only a few stations are available for shorter durations, such as 17 hours, 16 hours, or 13 hours per day. Influence on the cluster model: The "access time" feature might not carry as much weight as other features like location and number of chargers. If the K-means algorithm uses access time as a significant feature, it could overemphasize its importance, especially when most stations are available 24/7.

Before applying K-means, consider normalizing or standardizing the access time values, especially if they range widely (e.g., from 24 hours to 13 hours). This will ensure the feature is appropriately scaled and doesn't disproportionately influence the clustering.

Multivariate analysis

Charging Access by stations across charger types

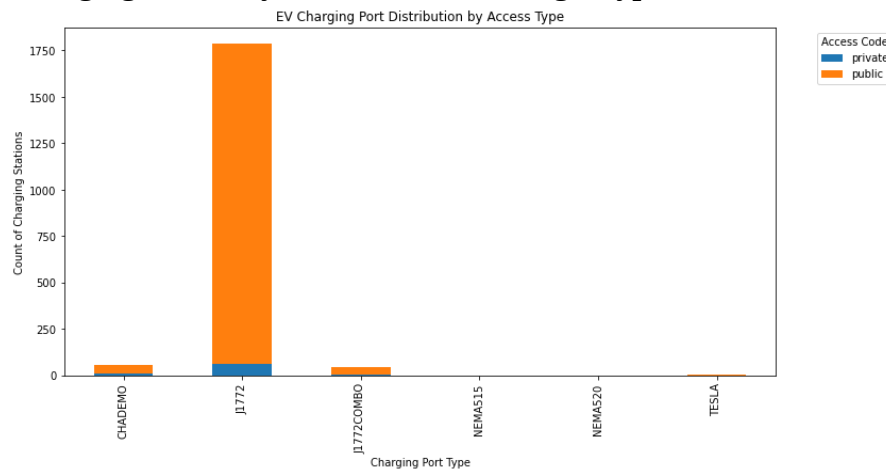


Fig.18: Bar plot charging stations and charging ports by access code.

From the plot, one thing stands out: J1772 dominates public charging, but it's also present in private settings just in smaller numbers. While J1772 is used in both public and private charging, its dominance in public spaces is clear. Private charging, on the other hand, is more diverse,

ranging from slower CHADEMO and J1772 Combo to NEMA outlets and other fleet chargers. This diversity in private charging stations reflects the flexibility of EV owners to install a variety of chargers based on their specific needs and vehicle requirements. In contrast, the public charging network tends to standardize around the J1772 connector to ensure broad compatibility and accessibility across a wide range of electric vehicles.

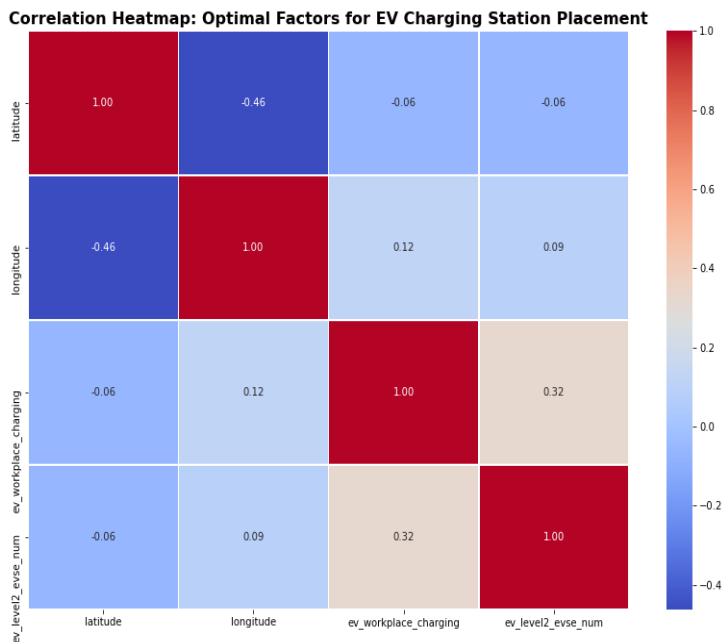


Fig.19: Correlation matrix

This heat-map reveals the relationships between geographic location, workplace charging availability, and Level 2 charging outlets—critical for effective EV infrastructure.

Geography’s Minimal Impact: Latitude and longitude have little influence on charging station numbers (ranging from -0.06 to 0.12), suggesting that infrastructure is more dependent on demand and policy than on location itself.

Workplace Charging Drives Expansion: A moderate positive correlation (0.32) between workplace charging and Level 2 outlets indicates that regions supporting workplace charging tend to foster broader EV infrastructure.

Subtle Eastward Trend: A slight positive correlation between longitude and charging availability (ranging from 0.09 to 0.12) hints at marginally higher infrastructure density as you move eastward, though the effect is minimal.

Bottom Line: Geography is not a limiting factor. To maximize impact, focus on areas with growing workplace charging networks, as these regions serve as natural catalysts for expanding public EV charging infrastructure.

Modelling

This section explores various unsupervised clustering models to determine the most effective algorithm for segmenting the Los Angeles dataset. A comprehensive evaluation is conducted across several popular clustering techniques, including K-means, DBSCAN, Gaussian Mixture Models (GMM), and Mean Shift Clustering. Each model undergoes fine-tuning and is assessed based on its capacity to identify meaningful patterns within the data.

The primary objective is to identify the best-performing clustering model for subsequent application to the Nairobi dataset. Upon selection, this model will be implemented on Nairobi's data, and its performance will be evaluated compared to the results of the Los Angeles dataset.

In addition to model performance, key contextual factors influencing the placement of electric vehicle (EV) charging stations in Los Angeles are considered. This includes an examination of geographical, infrastructural, and demographic variables that could affect clustering outcomes and, consequently, the model's success in varying urban environments.

The overarching goal is to derive insights into the most suitable clustering technique for urban EV infrastructure planning and refine the methodology for future applications in other cities, such as Nairobi.

The `create_multicolumn_weights` function was applied in the preceding cells to address the class imbalance of key features. By assigning weights to these imbalanced features, this approach ensures that the minority class is given equal consideration as the majority class during model training. This helps reduce bias, resulting in more balanced and fair predictions.

The columns `access_code_cleaned`, and `ev_workplace_charging` were dropped due to their significant class imbalance, as their inclusion could introduce bias into the model. Given their potential to affect predictions toward the majority class, removing them disproportionately was deemed necessary to improve model fairness. This ensures that imbalanced features do not influence the model's overall performance.

Additionally, several features were scaled, including latitude, longitude, access hours, `ev_level2_else_num`, and all frequency-encoded variables. Scaling these features ensures that each contributes equally during model training. Without scaling, features with differing ranges and units could dominate the model, negatively affecting performance. Normalizing these features enhances both model performance and convergence speed.

K=2: Silhouette Score = 0.8951, Davies-Bouldin Index = 0.1867, Calinski-Harabasz Index = 24692.25
 K=3: Silhouette Score = 0.9254, Davies-Bouldin Index = 0.1675, Calinski-Harabasz Index = 116323.77
 K=4: Silhouette Score = 0.9496, Davies-Bouldin Index = 0.1479, Calinski-Harabasz Index = 558561.34
 K=5: Silhouette Score = 0.9597, Davies-Bouldin Index = 0.1966, Calinski-Harabasz Index = 2778507.93
 K=6: Silhouette Score = 0.9729, Davies-Bouldin Index = 0.1197, Calinski-Harabasz Index = 6461600.90
 K=7: Silhouette Score = 0.9772, Davies-Bouldin Index = 0.1461, Calinski-Harabasz Index = 28096838.42
 K=8: Silhouette Score = 0.9809, Davies-Bouldin Index = 0.1094, Calinski-Harabasz Index = 43595465.98
 K=9: Silhouette Score = 0.9814, Davies-Bouldin Index = 0.1485, Calinski-Harabasz Index = 65060950.08
 K=10: Silhouette Score = 0.7987, Davies-Bouldin Index = 0.2436, Calinski-Harabasz Index = 78858742.09

Using k=5 for final clustering.

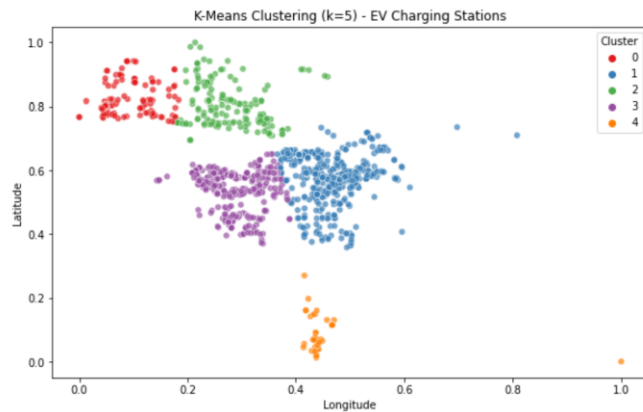
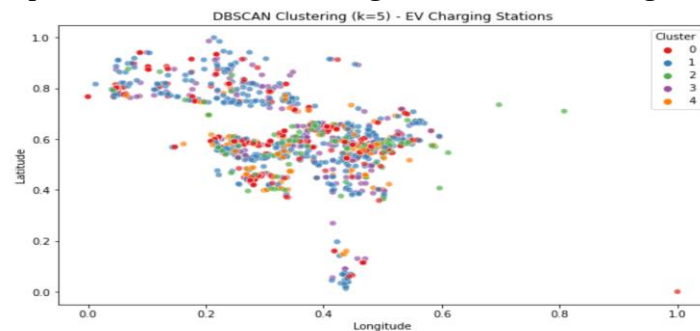


Fig.20: KMeans Clustering

KMeans results show improved clustering quality with higher K values:

- Silhouette Score increases from 0.8951 at K=2 to 0.9814 at K=9, peaking at K=8.
- Davies-Bouldin Index decreases, reaching its lowest at K=6 (0.1197) before rising at K=10 (0.2436).
- Calinski-Harabasz Index sharply rises, peaking at K=10 (78,858,742).

K=5 was chosen for final clustering, balancing the high silhouette score and reasonable separation before diminishing returns or overfitting. Next, we will test other clustering models.



DBSCAN Clustering Metrics:
 Silhouette Score: 0.8696
 Davies-Bouldin Index: 0.1283
 Calinski-Harabasz Index: 119618658.9850

Fig.21: DBSCAN Clustering

The DBSCAN results showed good clustering performance: Silhouette Score (0.8696): Indicates well-separated and coherent clusters. Davies-Bouldin Index (0.1283): Low value suggests distinct, non-overlapping clusters. Calinski-Harabasz Index (119,618,658.9850): High value indicates excellent separation and compactness. DBSCAN produced well-defined clusters, but its performance depends on parameter choices like epsilon and min_samples.

K=3: Silhouette Score = 0.9227, Davies-Bouldin Index = 0.1744, Calinski-Harabasz Index = 85591.63
K=5: Silhouette Score = 0.9597, Davies-Bouldin Index = 0.1966, Calinski-Harabasz Index = 2778507.93
K=7: Silhouette Score = 0.9489, Davies-Bouldin Index = 0.2446, Calinski-Harabasz Index = 8609175.86
K=9: Silhouette Score = 0.7983, Davies-Bouldin Index = 0.2192, Calinski-Harabasz Index = 45197895.62

Using best k=5 for final clustering.

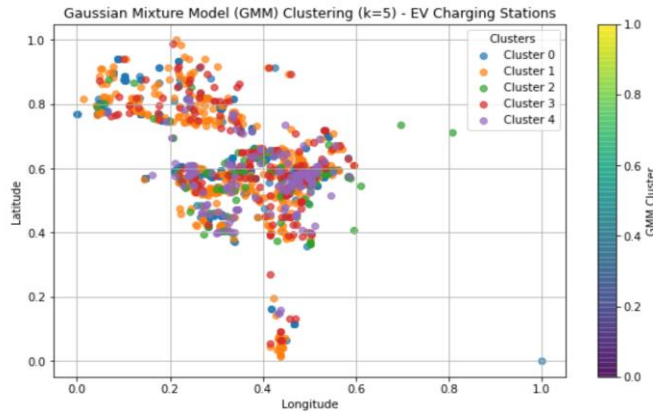


Fig. 22: GMM Clustering

The choice of K=5 for final clustering was based on several key metrics. The Silhouette Score of 0.9597 indicated well-defined and distinct clusters, while the Davies-Bouldin Index of 0.1966 suggested good separation with minimal overlap. Additionally, the Calinski-Harabasz Index of 2,778,507.93 showed strong separation and compactness. Despite K=3 having a slightly better Silhouette Score, K=5 provided a more balanced clustering solution across all metrics, offering better overall performance.

The clustering models worked effectively with the Los Angeles dataset, producing meaningful clusters aligned with our expectations. However, adjustments and fine-tuning of the clustering algorithms will be necessary to ensure robustness and generalizability when applied to the Nairobi dataset, considering factors such as population and power.

Given the potential differences in data characteristics, like distribution, density, and features specific to Nairobi, tweaking parameters such as the number of clusters, distance metrics, or initialization methods will be crucial to optimize performance.

Once the models are fine-tuned for Nairobi, we expect to identify clusters representing potential locations for new EV charging stations. Analyzing these clusters should allow us to map at least 10 viable locations, considering accessibility, demand, and geographical distribution. This process will inform future planning for expanding EV infrastructure in Nairobi, ensuring stations are placed strategically in areas with the highest demand.

The Best Model

While GMM showed the best performance in terms of clustering quality, KMeans was selected due to its advantages in spatial clustering. GMM is density-based, clustering based on Gaussian distributions and overall feature similarity rather than strictly on distance. This makes it more flexible but less suitable for precise spatial separation scenarios. On the other hand, KMeans minimizes Euclidean distance, which is more appropriate for spatially defined clusters.

Additionally, KMeans scales well in high-dimensional spaces and can handle large datasets efficiently, making it the preferred choice for this analysis.

EV Charging Stations in Nairobi

In this section, we focus on the strategic placement of electric vehicle (EV) charging infrastructure across Nairobi to support the growing adoption of EVs. Key considerations include high-traffic areas, residential neighborhoods, malls, supermarkets, offices, and public transport hubs. The goal is to position charging stations in locations with high vehicle flow and population density, ensuring they are convenient and accessible and provide efficient service for EV owners. Data-driven analysis, utilizing tools such as the Google Maps API, will play a crucial role in identifying optimal locations, contributing to developing a reliable and sustainable EV charging network in Nairobi.

Fetching EV Charging Stations Using Google Maps API

Fetching EV charging stations using the Google Maps API involves extracting location-based data from key areas of interest, such as malls, petrol stations, public parking, and bus terminals. By leveraging Google's Places API, we can gather information about these locations within a specified radius, considering factors like traffic patterns, population density, and access to the local power grid.

This data-driven approach enables the identification of strategic areas for new EV charging stations, ensuring they are placed in high-traffic, underserved, and densely populated regions. To retrieve relevant data, we make a pull request to search for locations using the keyword "EV charging station." The API filters and returns results that match the search criteria.

The response returned 19 results, providing detailed information about existing charging stations, including their locations, user ratings, and precise geographical coordinates. This data is invaluable for identifying areas where EV stations are already established, allowing for more effective planning and optimization of future station placements.

Fetching Traffic Data for EV Stations

To optimize the placement of EV charging stations, a pull request was made to retrieve traffic data for specific locations using the Google Maps API. The API provides real-time information on road congestion, traffic volume, and average travel times by passing coordinates and querying traffic patterns. This data is crucial for assessing the accessibility of EV charging stations, ensuring they are located in high-traffic areas with significant vehicle flow, thereby optimizing both visibility and usage. However, the pull request did not return the expected real-time traffic data, indicating that further investigation may be needed. To obtain reliable traffic data for the intended locations, alternative methods or secondary data sources may also need to be explored.

Clustering the Existing Ev charging stations

The model faced challenges in making structured predictions due to insufficient features like live traffic data, power grid availability, and vehicle population density. With minimal data, the model struggled to generate accurate insights for optimal EV charging station placement. To improve, we can start by plotting the existing charging stations to identify gaps and underserved areas. Focusing on high-traffic locations in Nairobi County, such as shopping malls, petrol stations, office buildings, and public transport hubs, will help guide the placement of new stations in areas with significant vehicle flow and foot traffic.

Extracting additional data, like the locations of shopping malls and other key establishments, will further refine our predictions. These locations typically have high foot traffic and long dwell times, making those ideal spots for EV charging stations. Without real-time traffic or power grid data, we can use these points of interest to predict where new stations are most needed, ultimately improving the coverage and accessibility of EV infrastructure across Nairobi.

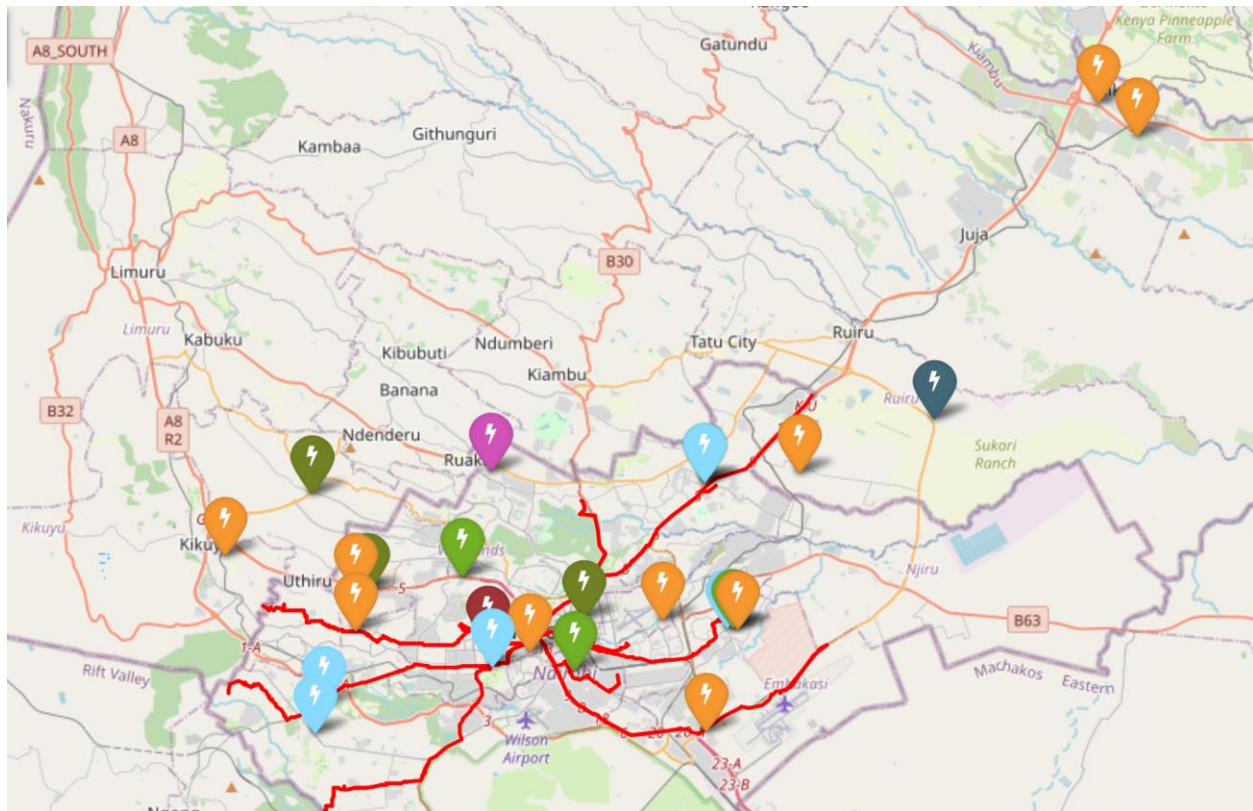


Fig.23: Distribution of Existing Ev charging stations

1. Yellow - the suggested charging stations are purely based on underserved areas and existing Ev charging stations infrastructure.
2. Blue - BasiGo charging stations.
3. Green - EVchaja charging stations.
4. Yellow - Ampersand charging stations.
5. Brown - Electric Vehicle charging station.

To effectively map new EV charging stations in Nairobi, a strategic approach should focus on key points of interest (POIs) and underserved areas. High-traffic POIs such as shopping malls, petrol stations, office buildings, and public transport hubs are ideal locations for charging stations, given their frequent traffic. These locations typically offer high visibility and accessibility, making them well-suited for EV infrastructure.

Below, we make API pull requests to Google to obtain a list of POIs in Nairobi. This data will be used to map potential EV charging stations, focusing on underserved and high demand areas.

Suggested charging stations along access roads and highways by adjusting their locations using OSMNx nearest road nodes.

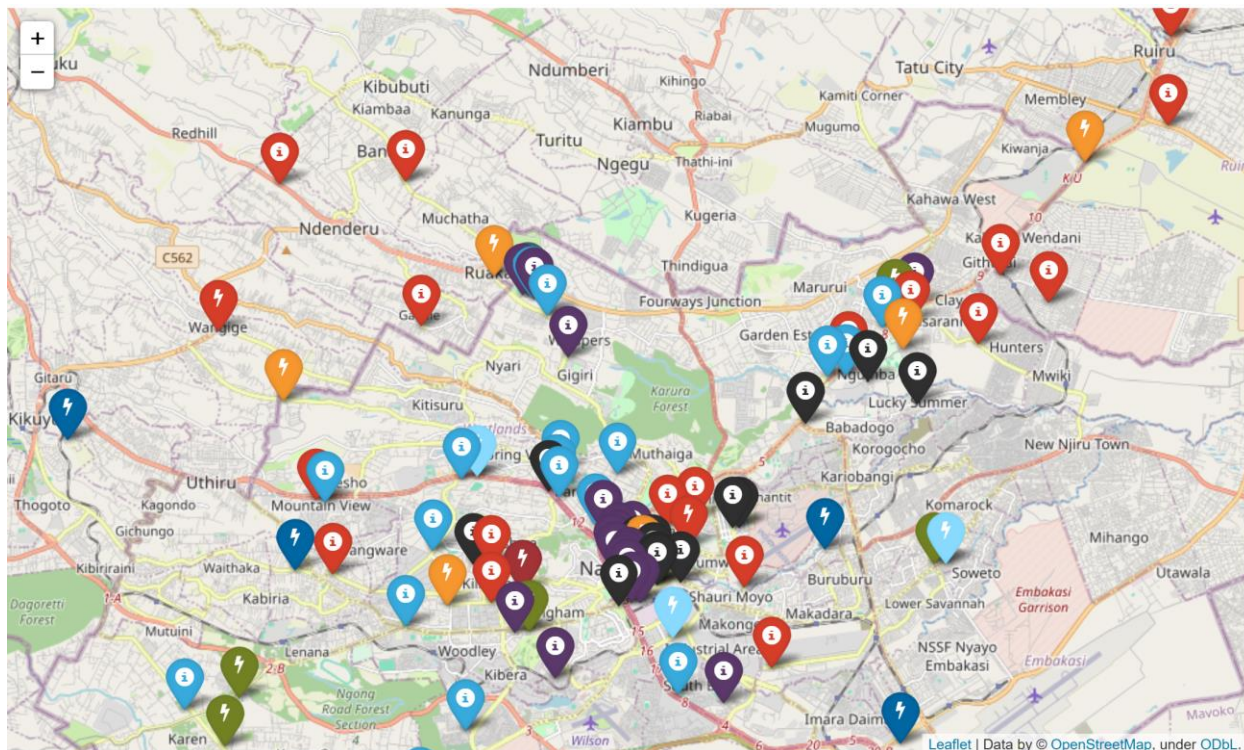


Fig. 24: Distribution of Existing Ev charging stations

Conclusions

The model was able to successfully predict ten charging stations based on both points of interest and underserved areas. We successfully achieved our objective by developing an AI-powered platform that accurately identifies optimal EV charging station locations through the integration of real-time usage data, geospatial analysis, and traffic patterns.

Despite facing constraints in obtaining reliable traffic data, vehicle population density, and KPLC power grid information, the model effectively addresses underserved areas and aligns with regional infrastructure needs, ensuring scalability and sustainability. This platform provides an efficient, data-driven solution for expanding EV charging infrastructure.

Recommendations

1. **Incorporate Underserved Residential Areas into the Model:** Enhance the mapping model by including highly modern populated residential neighborhoods in the suburbs of Nairobi. This will ensure the model identifies regions with limited charging infrastructure, allowing for optimized placement of new stations in underserved areas.
2. **Optimize Station Placement Based on Traffic and Accessibility Data:** Integrate real-time traffic patterns and road accessibility into the model. Prioritize high-traffic roads, like the Waiyaki Way and Mombasa Road, to ensure the model suggests charging stations at easily accessible and strategic points. This approach will improve station coverage and reduce charging time for users by placing stations along major commuter routes.
3. **Factor in Power Grid Access and Sustainable Energy Solutions:** Refine the model by integrating data on power grid access and incorporating sustainable energy options, like solar charging stations. This will enable the model to recommend locations that meet demand and are supported by reliable and sustainable energy sources, ensuring the charging network's efficient operation and long-term viability.

Next Steps

1. **More data Integration:** The first step is to gather and integrate a range of data, including real-time EV usage patterns, geospatial data, traffic flow, access to the power grid, and sustainability options like solar energy potential. With this data, we will fine-tune machine learning algorithms and geospatial analysis tools to make the most accurate predictions for where EV charging stations should be located.
2. **Deployment to a cloud platform like AWS, google Cloud Platform (GCP) or Microsoft Azure**
3. **Testing and Evaluation:** Implementing the platform in a select region like Nairobi will test the model's real-world effectiveness. Based on the model's recommendations, charging stations will be deployed, and we will collect real-time data on usage, traffic, and sustainability to evaluate the platform's performance.
4. **Stakeholder Collaboration:** Ensure the platform aligns with regional policies and infrastructural requirements. Collaborate with energy providers, local authorities, and other stakeholders.

References

Siemens Stiftung. (2023). *Unlocking the growth potential of Kenya's e-mobility sector*. Siemens Stiftung. <https://www.siemens-stiftung.org/wp-content/uploads/2030/09/studie-unlocking-the-growth-potential-of-Kenya-E-Mobility-Sector.pdf-1.pdf>

Capital FM. (2024, September). Lack of infrastructure hinders EV adoption in Kenya. *Capital FM*. <https://www.capitalfm.co.ke/business/2024/09/lack-of-infrastructure-hinders-ev-adoption-in-kenya/>